

TASK 1

Q1) As the labels contain 10 unique classes, we set $K = 10$ for all runs. The final SSEs from the implementation are 391129 for Euclidean K-means, 2462 for Cosine K-means and 5508 for Jaccard K-means. Cosine K-means produces the lowest SSE which means the clusters formed are very compact followed by Jaccard K means and the Euclidean version has the largest SSE. Hence Cosine K-means is the best method in terms of SSE on this dataset.

```
Converged at iteration 80
Euclidean Distance Metric Results:
Final SSE: 391129.7599267777
Number of iterations: 81
Time taken: 31.9449 seconds
```

```
Converged at iteration 40
Cosine Similarity Distance Metric Results:
Final SSE: 2462.8528582404065
Number of iterations: 41
Time taken: 19.8196 seconds
```

```
Converged at iteration 4
Generalized Jaccard Distance Metric Results:
Final SSE: 5508.756354877965
Number of iterations: 5
Time taken: 37.3528 seconds
```

Q2) Using majority vote labels to map the clusters to classes, the predictive accuracies are approximately 60% for Euclidean K-means, 62% for Cosine K-means and 50% for Jaccard K-means. We can see that the Cosine K-means achieves the highest accuracy, this indicates that angle based similarity between the data points provides a more effective notion of closeness for this dataset than either Euclidean or Jaccard.

```
Converged at iteration 80
Cluster to Label Mapping (Euclidean): {0: np.int64(1),
Euclidean Distance Metric Accuracy: 60.44%
```

```
Converged at iteration 40
Cluster to Label Mapping (Cosine): {0: np.int64(1),
Cosine Similarity Distance Metric Accuracy: 62.43%
```

```
Converged at iteration 4
Cluster to Label Mapping (Jaccard): {0: np.
Jaccard Distance Metric Accuracy: 50.39%
```

Q3) With the stopping rule (no centroid change, SSE increase, or reaching max iteration limit) Euclidean K means required the most iterations to converge, it reaches the maximum of 50 iterations, while Cosine K-means converges in 41 iterations and Jaccard K means converges only in 5 iterations. But Jaccard consumes the most time

because each iteration is computationally expensive whereas Euclidean and Cosine distances are cheaper to calculate.

```
1. Running Euclidean K-means (max_iter=50)...  
Stopped after reaching maximum iterations: 50  
  Iterations: 50  
  Time: 19.74 seconds  
  Final SSE: 391196.60
```

```
2. Running Cosine K-means (max_iter=50)...  
Converged at iteration 40  
  Iterations: 41  
  Time: 19.68 seconds  
  Final SSE: 2462.85
```

```
3. Running Jaccard K-means (max_iter=50)...  
Converged at iteration 4  
  Iterations: 5  
  Time: 36.89 seconds  
  Final SSE: 5508.76
```

Convergence Results:

Method	Iterations	Time (seconds)
Euclidean	50	19.74
Cosine	41	19.68
Jaccard	5	36.89

Most iterations required: Euclidean (50 iterations)
Most time required: Jaccard (36.89 seconds)

Q4) For all three terminating conditions, Cosine K-means consistently achieves the lowest SSE 2462 and Jaccard K-means has a moderate SSE (5508) and Euclidean K-means has the highest SSE. The SSE values for Cosines and Jaccard are essentially unchanged across the three stopping rules, while Euclidean shows only a small increase when run for maximum number of iterations. This means Cosine K-means gives the most compact clusters and is most robust to the choice of termination criterion.

1. Euclidean (no centroid change):

Converged (no centroid change) at iteration 81
SSE: 391129.76, Iterations: 81

2. Cosine (no centroid change):

Converged (no centroid change) at iteration 41
SSE: 2462.85, Iterations: 41

```
3. Jaccard (no centroid change):  
   Converged (no centroid change) at iteration 5  
   SSE: 5508.76, Iterations: 5
```

```
1. Euclidean (SSE increase):  
   Reached max iterations: 100  
   SSE: 391129.76, Iterations: 100  
  
2. Cosine (SSE increase):  
   Reached max iterations: 100  
   SSE: 2462.85, Iterations: 100  
  
3. Jaccard (SSE increase):  
   Reached max iterations: 50  
   SSE: 5508.76, Iterations: 50
```

```

1. Euclidean (max iterations):
   Reached max iterations: 50
   SSE: 391196.60, Iterations: 50

2. Cosine (max iterations):
   Reached max iterations: 50
   SSE: 2462.85, Iterations: 50

3. Jaccard (max iterations):
   Reached max iterations: 50
   SSE: 5508.76, Iterations: 50

```

Distance Metric	Condition 1 (No Centroid Change)	Condition 2 (SSE Increase)	Condition 3 (Max Iter=100)
=====	=====	=====	=====
Euclidean	391129.76	391129.76	391196.60
Iterations:	81	100	50
Cosine	2462.85	2462.85	2462.85
Iterations:	41	100	50
Jaccard	5508.76	5508.76	5508.76
Iterations:	5	50	50

Q5) For this dataset, Cosine K-means gives the best overall performance. It achieves lowest SSE, highest accuracy and converges in a reasonable number of iterations, this proves that angle based similarity is better for high dimensional data. Euclidean K means is simple and very fast but gives a very large SSE value and Jaccard K-means is computationally very expensive. Thus this shows that choice of distance metric and centroid definition affects both clustering quality and runtime.

TASK 2

Q3) c) Across the three models, PMF achieves the lowest average RMSE and MAE scores, indicating that the latent-factor method captures user-item interactions more effectively than other memory based approaches. The user based collaborative filtering performs worse and item based collaborative filtering shows the highest error of the bunch. This pattern occurs because PMF can generalize from sparse rating data.

Evaluating RMSE, MAE of algorithm KNNBasic on 5 split(s).

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std
RMSE (testset)	0.9688	0.9659	0.9677	0.9666	0.9667	0.9671	0.0010
MAE (testset)	0.7442	0.7433	0.7457	0.7439	0.7418	0.7438	0.0013
Fit time	0.10	0.11	0.13	0.12	0.11	0.11	0.01
Test time	1.00	1.34	1.23	0.98	0.94	1.10	0.16

Evaluating RMSE, MAE of algorithm KNNBasic on 5 split(s).

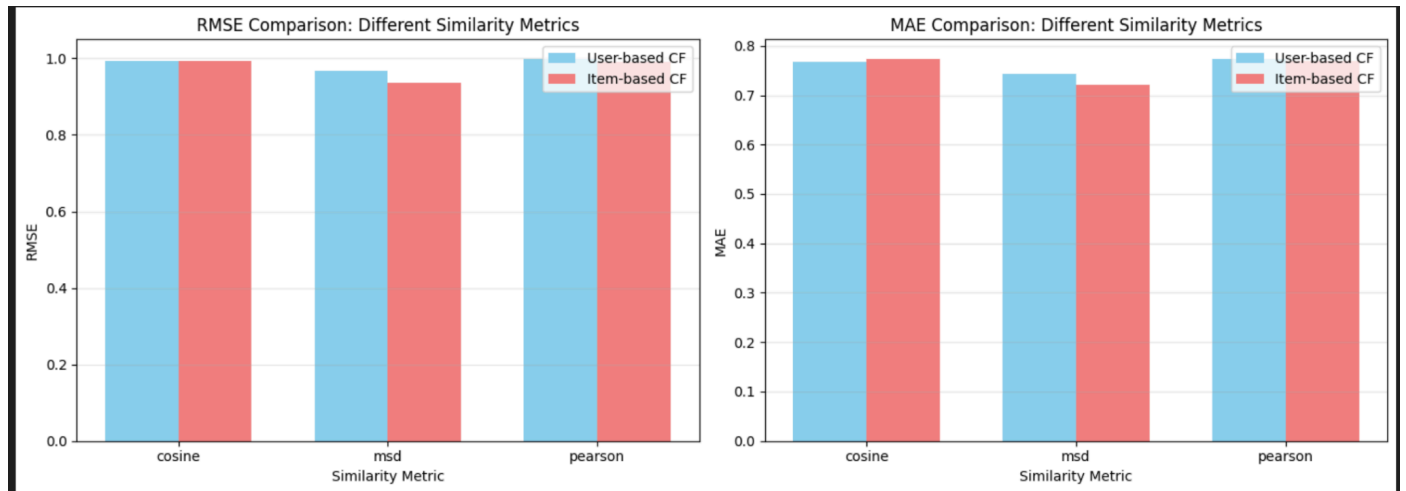
	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std
RMSE (testset)	0.9388	0.9365	0.9385	0.9310	0.9326	0.9355	0.0032
MAE (testset)	0.7240	0.7232	0.7217	0.7160	0.7218	0.7213	0.0028
Fit time	3.61	3.50	3.57	3.54	3.49	3.54	0.05
Test time	4.34	4.53	4.37	4.55	4.36	4.43	0.09

Evaluating RMSE, MAE of algorithm SVD on 5 split(s).

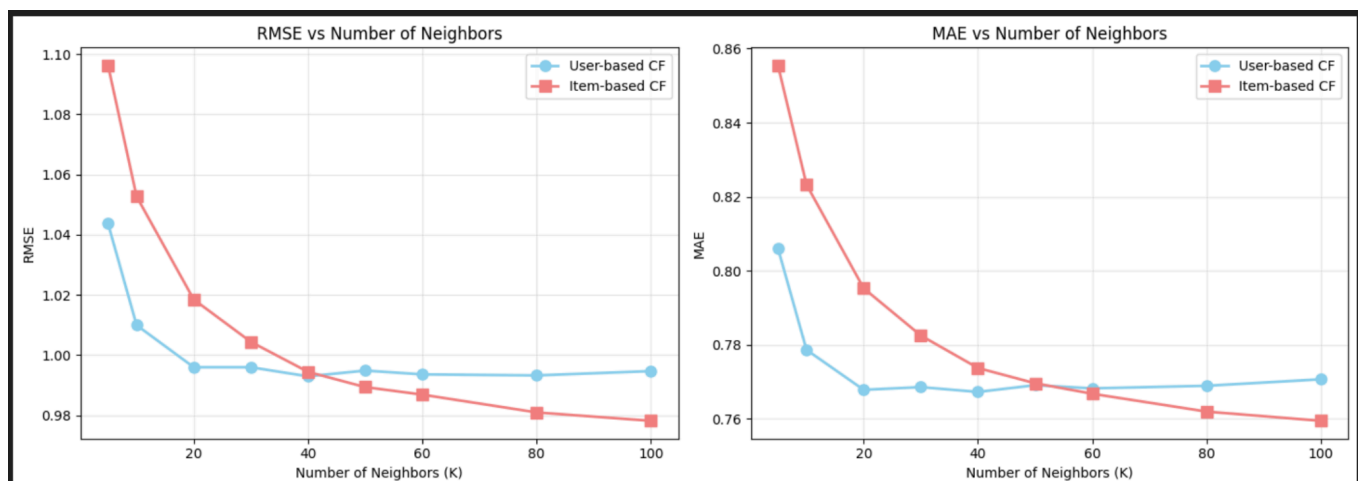
	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	Std
RMSE (testset)	0.8937	0.9007	0.8937	0.8845	0.9011	0.8947	0.0060
MAE (testset)	0.6889	0.6909	0.6889	0.6828	0.6928	0.6888	0.0034
Fit time	0.95	0.92	0.95	0.88	0.92	0.92	0.03
Test time	0.21	0.08	0.20	0.08	0.20	0.16	0.06

d) While comparing average RMSE and MAE values for the three methods, PMF again is the strongest model, it achieves lowest error among the approaches. User based collaborative filtering performs moderate and item based collaborative filtering shows high error, this indicates that item similarity is less reliable for predictions on this dataset. Thus PMF is the best model as it captures latent user item factors.

e) For both the user based and item based collaborative filtering, MSD gives the lowest RMSE and MAE while cosine and Pearson perform worse and are close to each other. This means that measuring similarity via squared rating differences leads to more accurate predictions than cosine based or correlation based on this dataset. The ordering of the three metrics is consistent for both user based and item based CF.



f) Varying the number of neighbors (K) has a clear impact on both the user based and item based collaborative filtering. For user based CF, RMSE decreases initially as K increases and reaches its lowest value around moderate neighborhood sizes, after which performance begins to flatten or get worse. In contrast, item based CF benefits more steadily from larger K values, with RMSE continuing to decrease as K grows. This happens because item similarity becomes more reliable when aggregated across larger set of neighbors.



g) For user based collaborative filtering, the best K value is the value that gives us the minimum RMSE, which in my results occurs around $K = 40$, while item based collaborative filtering achieves its lowest RMSE at $K = 100$. This shows that the optimal neighborhood size for users is much smaller than for items. This is because user similarity saturates with a moderate number of neighbors while item similarity keeps improving as more neighbors are included. Thus the best K for user based CF is not best K for item based CF.

Code link:

<https://github.com/AtharvaBOT7/CSE572-DM-HW3-1233380696>